

Altec Website Rebuild

Architecture & Content Management Proposal — Draft for Review

Prepared 2026-07-14 · Status: draft, open for edits

Contents

1. Project Summary
2. Business Requirements Recap
3. Design Reference — What We're Borrowing From Viessmann.it
4. Technology Stack
5. System Architecture
6. Data Model
7. Admin CMS — Feature List
8. Public Site — Proposed Page Structure
9. Hosting & Deployment
10. Open Questions & Decisions Needed
11. Suggested Next Steps

1. Project Summary

Altec's current website (altec-al.com) is a WordPress site. The goal is to rebuild it as a modern, self-managed website — without WordPress — that the owner can update himself as new products and projects come in, running entirely on the existing shared cPanel hosting with no external/cloud services.

2. Business Requirements Recap

- **Core business:** AC (air conditioning) sales and installation is the owner's primary activity. Reconstruction and home furnishing ("arredamento") are secondary, occasional services — the new site's structure should reflect that weighting, unlike the current site which gives all services/products roughly equal billing.
- **Partnership:** the owner collaborates with **Viessmann** (heating/AC/energy systems manufacturer) — this relationship should be visible on the site (branding, product lines carried).
- **Ongoing content growth:** products, gallery items, and news will be added continuously — this is the reason we're building a real CMS rather than a static, hand-coded site.

- **Multi-language:** current site supports English/Albanian (`?lang=`); the new site should keep at least these two languages, managed from the CMS.
- **No e-commerce:** products are informational/catalog-style, not a shopping cart. A working **contact/quote-request form** should be added — the current site conspicuously lacks one despite listing extensive contact details.
- **Hosting constraint:** must run entirely on the existing shared cPanel server. No external databases, cloud CMS, or third-party hosted services.

3. Design Reference — What We're Borrowing From Viessmann.it

The owner specifically likes viessmann.it's structure. At Altec's scale, the useful patterns are:

- **Hero + promo sections** on the homepage (current offers, featured products/brands)
- **Card-based product categories** rather than long text lists
- **Solution-based grouping** — organize by use-case (e.g. "For Homes" / "For Offices & Commercial") in addition to flat product categories
- **References/case studies gallery**, segmented by project type (matches the existing "Works" portfolio concept)
- **A news/blog section** for updates and completed projects

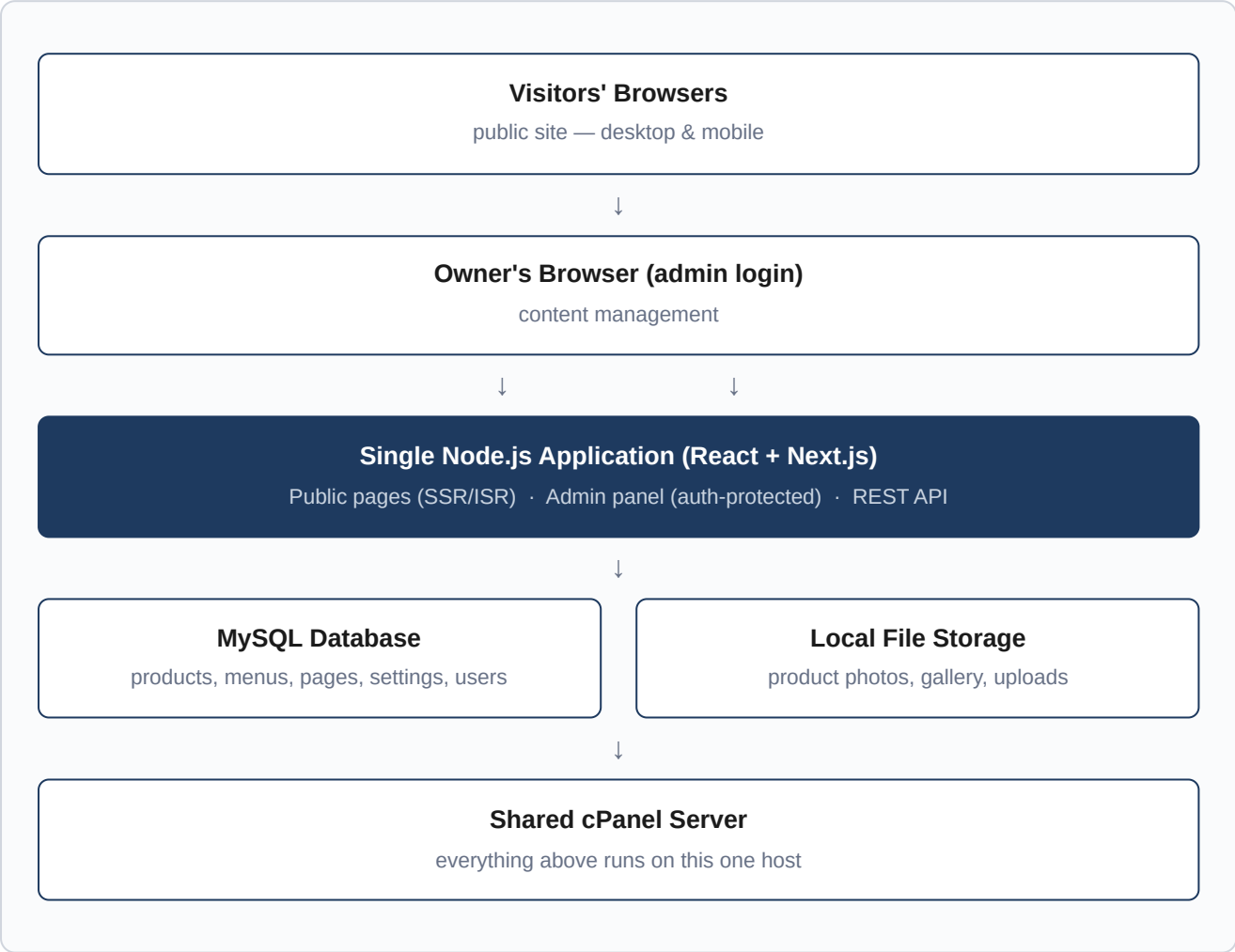
Deliberately **not** replicating (too large-scale/B2B2C for a single local installer): partner-installer locator portal, financing simulators, training academy, restricted dealer portal.

4. Technology Stack Confirmed

Layer	Choice	Notes
Frontend (public site)	React (Next.js)	Server-rendered / statically generated pages for SEO — important for a local business relying on search visibility.
Admin / CMS panel	React	Either a protected section of the same Next.js app (<code>/admin</code>) or a separate React app — see Section 9.
Backend / API	Node.js	Express, Fastify, or Next.js API routes — serves both the public site and the admin panel.
Database	MySQL / MariaDB	Chosen over MongoDB specifically because cPanel bundles MySQL natively at no extra cost — avoids any external database dependency (e.g. MongoDB Atlas). Access via an ORM (Prisma, Sequelize, or Knex) for schema management and migrations.
Image/file storage	Local disk on the cPanel server	Uploaded via the admin panel, served directly from the server — no external CDN/storage service.

Hosting	Existing shared cPanel account	Single self-contained deployment — see Section 9 for the open dependency on Node.js App Manager support.
---------	--------------------------------	--

5. System Architecture



Recommendation: run this as **one Next.js application** that contains the public site, the admin panel, and the API — rather than three separate apps. Reasoning: shared cPanel Node.js hosting typically limits how many separate Node "applications"/ports you can run, so consolidating into one process keeps deployment simple. This can be split apart later if the project outgrows it.

6. Data Model

Core entities the CMS needs to manage. Field lists are indicative, not final.

Entity	Key Fields	Notes
--------	------------	-------

Page	slug, title (per language), content blocks, SEO meta	About Us, Partners, etc. — flexible content pages
MenuItem	label (per language), link/target, parent_id, sort_order, visible	Self-referencing for nested menus, owner-editable order
Category	name (per language), type (service / product), parent_id	Powers "Services" and "Products" groupings
Product / Service	name, description (per language), category_id, brand (e.g. Viessmann), images, published	Same underlying model can serve both products and services
GalleryItem / WorkItem	title, project_type, images/video, description, date	Portfolio / "Works" section, filterable by project type
NewsPost	title, body, cover_image, published_at, language	Blog/news/updates section
Setting	key, value	Key-value store for contact info, social links, homepage promo text, etc.
ContactSubmission	name, email, phone, message, created_at	Stores inquiries from the new contact/quote form
AdminUser	email, password_hash, role	Starts with a single admin (the owner); can expand to roles later
Language	code (en/al), label	Drives which translations are shown/editable

7. Admin CMS — Feature List

- Secure login (single admin account to start)
- **Products & Services:** create/edit/delete, assign to category, upload photos, mark published/unpublished, per-language text
- **Menu builder:** add/remove/reorder menu items, nest submenus, point items at pages or categories
- **Gallery/Works:** add projects with photos, tag by project type
- **News:** write/publish/edit posts
- **Settings:** contact details, social links, homepage promotional content
- **Contact form submissions:** a simple inbox view of inquiries received through the site
- **Language switcher** while editing, to fill in EN and AL versions of each piece of content

8. Public Site — Proposed Page Structure

1. **Home** — hero/promo, featured AC products & Viessmann partnership, quick links into Services/Products, recent projects, recent news
2. **AC Sales & Installation** (*primary section*) — products by category, brands carried (incl. Viessmann), "solution" groupings (residential / commercial)
3. **Reconstruction & Home Furnishing** (*secondary section*)
4. **About Us**
5. **Partners** (Viessmann and others)
6. **Works / Gallery** — project portfolio, filterable by type
7. **News**
8. **Contact** — contact form (new) + office details/map

Every page above is driven by the menu builder and content entities from Section 6, so the owner can rename, reorder, add, or remove sections without a code change.

9. Hosting & Deployment

SELF-CONTAINED BY DESIGN

Every component — Node app, MySQL database, uploaded images — runs on the one existing shared cPanel account. No external accounts, subscriptions, or services required to operate the site.

- Node app deployed via cPanel's `Setup Node.js App` tool (Phusion Passenger)
- MySQL database created through cPanel's MySQL Database Wizard
- Uploaded images stored under the app's file directory, served directly by the Node app or via the web server
- Deployment via Git push / SSH / File Manager upload, depending on what the hosting account allows

10. Open Questions & Decisions Needed

BLOCKING — NEEDS VERIFICATION

Does the current cPanel plan have "**Setup Node.js App**" available (Software section of cPanel)? The entire architecture above assumes a persistent Node.js process is supported. If it isn't, we need to revisit hosting before writing code.

- Confirm the two languages to support at launch are English and Albanian only (or more later?)

- Should the contact form send email via the server's SMTP/PHP mail, and where should submissions be reviewed — only in the admin inbox, or also emailed to the owner directly?
- Do we need multiple admin accounts/roles (e.g. owner + an office assistant), or just one login?
- Any size/quantity expectations for product photos and galleries, to plan storage on the shared server?
- Should "Works/Gallery" support video, or photos only?
- Domain/DNS and SSL — staying on the current domain and cPanel account, or is anything changing there?

11. Suggested Next Steps

1. Review this document — add, remove, or adjust anything above
2. Confirm Node.js App Manager availability on the cPanel account
3. Finalize the data model (Section 6) and page structure (Section 8)
4. Scaffold the Next.js project structure and MySQL schema/migrations
5. Build the admin CMS core (auth, menus, settings) before content-type screens
6. Build out public page templates against the API